Fly GACA — Instructor Dashboard — Design Spec

SECTION 06-operations-it TYPE spec STATUS active OWNER Founder UPDATED 2026-06-16

Status: design, not built · **Maps to:** action plan §5.2, `roadmap.md` Phase 5 **Prepared:** 2026-05-25

This is the build spec for the B2B instructor/school dashboard. It is deliberately spec-first: the hard part is not the UI, it is a multi-tenant data model and the security rules that let one user see another's data **without** weakening the strict per-user isolation the platform has today. That has to be designed, not rushed.

`schools.html` already *sells* this — "a school admin dashboard … cohort exam-readiness and per-cadet progress … is included with every contract." This spec builds that promise.

---

1. The core constraint

Today the model is strict per-user isolation. `firestore.rules` says it plainly: *"There is no path by which one user can read another user's profile or logbook."* Every rule is `isOwner(uid)`.

The instructor dashboard requires **controlled cross-user reads** — an instructor seeing a cadet's progress. The entire design below exists to do that safely. Get the rules wrong and a cadet's records leak; that is the single biggest risk and the reason this is spec-first.

The key design decision that resolves it: instructors never read into `users/{cadetUid}`. Instead, each cadet's client writes a **scoped progress summary** into the school's own subtree (`schools/{id}/roster/{cadetUid}`). Instructors read the *school's* documents, never the cadets'. `users/{uid}` isolation stays 100% intact.

---

2. Roles

Role	Who	Capability
Cadet / pilot	exists today — `users/{uid}`	Owns their data; opts in to a cohort
Instructor	new	Reads their cohort's progress; assigns reading paths
School admin	new	Manages seats, instructors and the roster; sees the cohort rollup

A cadet on a school plan is already tagged: `users/{uid}.entitlement.schoolId` exists in the current model (`store.js`, for `entitlement.source === 'school'`). That field is the anchor for everything here.

3. Consent & privacy — first-class

A cadet's logbook and study data are personal data. The PDPL DPIA is already a launch gate (`roadmap.md`); cross-user visibility **must** be inside that DPIA's scope.

Two rules this spec commits to:

1. **Scope.** Instructors see **study and exam-readiness data only** — ground-school progress, mock-exam scores, currency flags. They do **not** see the personal logbook. The logbook is the pilot's own record; keep it private. (Open decision — §10 — but this is the recommended default.)
2. **Explicit, revocable consent.** When a cadet is enrolled into a cohort they see a consent screen naming exactly what the school will see, and must accept. They can revoke it from Settings, which flips their roster entry to `consent: false` and blanks the shared summary.

4. Data model (Firestore)

New top-level `schools` tree. Nothing under `users/{uid}` changes shape; one new subcollection is added there (the progress summary, also useful on its own).

```
schools/{schoolId}
  name              string
  adminUids         string[]      // school admins
  instructorUids    string[]      // instructors
  seatLimit         number        // seats purchased
  seatsUsed         number        // maintained by Cloud Function
  createdAt/updatedAt timestamps

schools/{schoolId}/roster/{cadetUid}
  // Enrollment fields — written ONLY by Cloud Functions:
  cadetName         string        // denormalised for the cohort list
  cadetEmail        string
  status            'invited' | 'active' | 'removed'
  consent           boolean       // the cadet accepted the data-sharing consent
  enrolledAt        timestamp
  // Progress summary — written by the CADET's own client (see §6):
  readiness         number        // exam-readiness %, 0-100
  lastMockScore     number
  lessonsDone       number
  currencyFlags     map           // { day:'ok', night:'due', ifr:'lapsed' }
  progressUpdatedAt timestamp
  // Instructor → cadet, written by instructors:
  assignedPaths     string[]      // reading-path ids the instructor suggests

users/{uid}/progress/summary        // NEW — the server-side progress doc
  readiness, lastMockScore, lessonsDone, currencyFlags, updatedAt
```

Prerequisite — Phase A. Study progress is currently **not** in Firestore — `store.js` persists only the profile and the logbook; mock-exam scores and lesson progress live in `localStorage` (`study-progress.js`). For an instructor to see anything, the cadet's progress must be server-side. So Phase A adds `users/{uid}/progress/summary`, written by the cadet's client. The cadet's client then also mirrors that summary into their `schools/{id}/roster/{cadetUid}` entry. Verify the current persistence before building.

---

5. Security rules — sketch

This is a sketch and must be unit-tested in the Firebase emulator before any deploy. It extends today's `firestore.rules` (the `users/{uid}` block is unchanged).

```
match /schools/{schoolId} {
  function school() {
    return get(/databases/$(database)/documents/schools/$(schoolId)).data;
  }
  function isAdmin() {
    return signedIn() && request.auth.uid in school().adminUids;
  }
  function isStaff() {
    return signedIn() && (request.auth.uid in school().adminUids
                       || request.auth.uid in school().instructorUids);
  }

  // The school doc: staff read; admins edit non-structural fields.
  // Creation and seat counts are Cloud-Function-only.
  allow read:   if isStaff();
  allow update: if isAdmin() && onlyAllowedSchoolFields();
  allow create, delete: if false;

  match /roster/{cadetUid} {
    function roster() { return resource.data; }
    // Staff see the whole cohort; a cadet sees only their own entry.
    allow read: if isStaff() || isOwner(cadetUid);
    // The cadet may update ONLY their own progress-summary fields, and only
    // while their consent is true. They can never touch enrollment fields.
    allow update: if isOwner(cadetUid)
                  && roster().consent == true
                  && onlyProgressFields();
    // An instructor may update ONLY assignedPaths.
    allow update: if isStaff() && onlyAssignedPaths();
    // Enrollment / removal is Cloud-Function-only (it also moves a seat
    // and sets users/{uid}.entitlement.schoolId).
    allow create, delete: if false;
  }
}
```

`onlyProgressFields()` / `onlyAssignedPaths()` / `onlyAllowedSchoolFields()` are diff checks — `request.resource.data.diff(resource.data).affectedKeys()` must be a subset of the allowed set. Write them explicitly; this is where a leak hides.

Why this is safe: instructors only ever read `schools/{id}/...`. There is still **no rule** anywhere that lets one user read another's `users/{uid}` tree. The cross-tenant surface is one collection, with a tight, testable rule set.

---

6. Enrollment & consent flow

1. **School created** — by Fly GACA on contract signing (a Cloud Function /admin tool), not self-serve. Sets `name`, `adminUids`, `seatLimit`.
2. **Admin adds instructors** — by email; a Cloud Function resolves the email to a uid and appends to `instructorUids`.
3. **Cadets enrolled** — admin enters cadet emails (recommended) **or** shares a school join-code. Either way a Cloud Function creates the `roster/{cadetUid}` entry with `status:'invited'`, `consent:false`.
4. **Cadet consents** — on next sign-in the cadet sees a consent screen: "Your flight school [name] will see your ground-school progress and mock-exam scores. They will not see your logbook. You can revoke this anytime in Settings." On accept → a Cloud Function sets `consent:true`, `status:'active'`, `entitlement.schoolId`, and increments `seatsUsed`.
5. **Revocation** — Settings → "Leave school cohort" sets `consent:false` and the client blanks the shared summary fields.

All seat/entitlement/roster-structure writes go through Cloud Functions (Admin SDK) — exactly as `entitlement` is handled today.

---

7. UI plan

New page `instructor.html` (gated to `isStaff`), plus a cadet consent surface.

- **Cohort view** — a table/cards: each cadet's name, exam-readiness %, last mock score, lessons complete, currency flags, last-active. Sort + filter (e.g. "show me who's behind"). This is the "see at a glance who is ready and who is falling behind" promised on `schools.html`.
- **Per-cadet drill-down** — the cadet's readiness trend, mock-exam history, ground-school lesson map, currency. Plus an **Assign reading paths** control → writes `assignedPaths`; the cadet sees them as suggested paths.
- **Admin view** — seats used / total, add/remove cadets and instructors, a cohort readiness rollup (average, distribution, count not-yet-started).
- **Cadet consent screen** — shown once on enrollment; re-summarised in Settings.

Reuse the existing `.curr-card`, `.stat-card`, account-page chrome and `gate.js`.

---

8. Build phases

Phase	Scope	Effort
A	Server-side progress summary (`users/{uid}/progress/summary`) written by the cadet client — useful on its own	S–M
B	`schools` + `roster` data model + security rules + emulator tests	M
C	Cloud Functions: create school, add instructor, enroll cadet, consent, revoke, seat accounting	M
D	`instructor.html` — cohort view + per-cadet drill-down	M
E	Admin seat management + reading-path assignment	M

Total ≈ **L** (a focused multi-week build, solo) — matches the action plan's estimate. Phases A and B are the foundation and must land first and correctly; D is the only part a user ever sees.

---

9. Open decisions (you)

1. **Logbook visibility** — instructors see study data only (recommended), or the logbook too? Privacy says study-only.
2. **Enrollment** — admin-enters-emails (more control, recommended) vs a join-code (less friction, weaker control)? Could support both.
3. **School creation** — Fly GACA-on-contract (recommended for a paid B2B product) vs self-serve signup.
4. **DPIA** — cross-user data sharing must be added to the PDPL DPIA scope. This is a launch gate; the instructor dashboard cannot go live until it's covered.

---

10. Risks

- **Security rules.** A rule mistake leaks cadet data. Mitigations: the no-reads-into-`users/{uid}` design above; explicit `affectedKeys()` diff checks; mandatory Firebase-emulator rule tests; staged rollout with one pilot school.
- **PDPL.** Cross-user visibility is exactly what a DPIA scrutinises — do not deploy before it's covered.
- **Stale summaries.** The cadet's client writes the progress summary; if it's stale the instructor sees old data. Mitigation: write the summary on every study session end, and show `progressUpdatedAt` in the UI so staleness is visible.
- **Scope creep.** Resist messaging, grading, attendance, billing in v1. v1 = see the cohort's readiness. That alone sells seats.